

**DUOC UC**

FullStack I

---

# **INFORME TÉCNICO**

## **Experiencia 3**

*Sistema de Gestión “Perlitas de Eucarion”*

### **Integrantes**

Yuliana Calvucura

Luis Grossolli

Catalina Salinas

Docente: Mariangeles Robinson

Fecha de entrega: 21/06/2026

# Índice

---

|                                                                |    |
|----------------------------------------------------------------|----|
| Índice                                                         | 2  |
| Introducción                                                   | 3  |
| 1. Contenido de la Experiencia 2 (detallado)                   | 3  |
| 1.1 Descripción general del proyecto                           | 3  |
| 1.2 Arquitectura de microservicios y patrón de capas           | 3  |
| 1.3 Modelo de datos relacional                                 | 3  |
| 1.4 Funcionalidades CRUD implementadas por microservicio       | 4  |
| 1.5 Manejo de excepciones                                      | 4  |
| 2. Diagrama actualizado                                        | 5  |
| Diagrama entidad-relación base de datos                        | 5  |
| 2.2 Diagrama de arquitectura de microservicios                 | 5  |
| 3. Servicio de autenticación                                   | 6  |
| 3.2 Autenticación por token                                    | 6  |
| 3.3 Validación del token en el Gateway                         | 8  |
| 3.4 Spring Security en cada microservicio: protección de rutas | 8  |
| 3.5 CORS                                                       | 9  |
| 3.6 Evidencia de funcionamiento (Postman)                      | 10 |
| 4. Implementación por capas                                    | 12 |
| 4.1 El patrón Controller-Service-Repository                    | 12 |
| 4.2 Ejemplo completo: microservicio de Materiales              | 12 |
| Capa Repository                                                | 12 |
| Capa Service                                                   | 12 |
| Capa Controller                                                | 13 |
| POST /materiales — crear                                       | 14 |
| GET /materiales — listar todos                                 | 15 |
| GET /materiales/{id} — buscar por id                           | 15 |
| PUT /materiales/{id} — actualizar                              | 16 |
| DELETE /materiales/{id} — eliminar                             | 17 |
| 5. Comunicación entre microservicios                           | 18 |
| 6. Swagger                                                     | 19 |
| 6.2 Implementación de microservicios en Spring Boot            | 19 |
| 6.3 Ejemplo en tres microservicios                             | 20 |
| 7. Testing                                                     | 21 |
| 7.2 Pruebas unitarias: el patrón AAA                           | 21 |
| 7.3 Herramientas utilizadas                                    | 22 |
| 7.4 Cobertura de testing del proyecto                          | 23 |
| Conclusión                                                     | 23 |
| Anexos                                                         | 24 |

# Introducción

---

El presente informe documenta el desarrollo de la Experiencia 3 del proyecto “Perlitas de Eucarion”, una aplicación basada en una arquitectura de microservicios construida con Spring Boot. El sistema centraliza la gestión de inventario (materiales y productos), ventas, pedidos, proveedores, gastos, calendario de producción, galería de imágenes y reportes de una pyme de bisutería, reemplazando un proceso anteriormente manual.

Sobre la base construida en la Experiencia 2, en esta etapa se incorporaron los microservicios faltantes hasta completar la propuesta de solución, un servicio de autenticación y autorización basado en tokens JWT, documentación de API mediante Swagger/OpenAPI, y un conjunto de pruebas unitarias con JUnit 5 y Mockito que cubre más del 60% de los microservicios del proyecto. A continuación se detalla cada uno de estos componentes, su justificación técnica y su implementación.

## 1. Contenido de la Experiencia 2 (detallado)

---

### 1.1 Descripción general del proyecto

“Perlitas de Eucarion” es una pyme de bisutería que requería digitalizar la gestión de su inventario, ventas, pedidos y producción. El proyecto resuelve esta necesidad mediante una arquitectura de microservicios en Spring Boot, donde cada dominio del negocio (materiales, productos, ventas, pedidos, categorías, proveedores, reportes, galería, gastos y calendario de producción) se implementa como un servicio independiente, con su propia base de datos, siguiendo el patrón Database-per-Service.

### 1.2 Arquitectura de microservicios y patrón de capas

Cada microservicio se construyó siguiendo el patrón CSR (Controller–Service–Repository):

- Controller: expone los endpoints REST, recibe las peticiones HTTP y delega la lógica al Service. No contiene lógica de negocio.
- Service: contiene la lógica de negocio y las reglas de validación. Orquesta llamadas al Repository y, cuando corresponde, a otros microservicios.
- Repository: interfaz que extiende JpaRepository y se encarga exclusivamente del acceso a datos mediante Spring Data JPA.

Esta separación de responsabilidades permite que cada capa sea testeada de forma aislada (ver sección 7) y facilita el mantenimiento, ya que un cambio en la lógica de negocio no debería afectar la forma en que se exponen los endpoints ni cómo se accede a los datos.

### 1.3 Modelo de datos relacional

El modelo de datos se diseñó de forma distribuida: cada microservicio posee su propio esquema (db\_materials, db\_products, db\_ventas, db\_orders, db\_categories, db\_suppliers, db\_reportes, db\_galeria, db\_gastos, db\_calendario y db\_seguridad\_perlita para el servicio de

autenticación), evitando que un microservicio acceda directamente a las tablas de otro. El detalle completo de entidades y relaciones se presenta en el diagrama entidad-relación de la sección 2.1.

## 1.4 Funcionalidades CRUD implementadas por microservicio

A continuación se resume el conjunto de operaciones disponibles en cada microservicio de negocio. Todas las rutas se consumen a través del Gateway (<http://localhost:9090>) o directamente por el puerto de cada servicio.

| Microservicio      | Puerto | Operaciones principales                                                                      |
|--------------------|--------|----------------------------------------------------------------------------------------------|
| materiales         | 8081   | GET, GET /{id}, GET /stock-bajo, POST, PUT, DELETE                                           |
| productos          | 8082   | GET, GET /activos, GET /{id}, GET /{id}/materiales, POST, POST /{id}/materiales, PUT, DELETE |
| ventas             | 8083   | GET, GET /{id}, GET /cliente/{id}, GET /resumen/metodo-pago, POST                            |
| pedidos            | 8085   | GET, GET /{id}, GET /estado/{estado}, GET /cliente/{id}, POST, PUT, DELETE, PATCH /estado    |
| categories         | 8084   | GET, GET /activas, GET /{id}, GET /obtenerProducto/{id}, POST, PUT, DELETE, PATCH /reasignar |
| proveedores        | 8086   | GET, GET /activos, GET /{id}, POST, PUT, DELETE, PATCH /activar, PATCH /desactivar           |
| reportes           | 8087   | GET, GET /{id}, POST, DELETE, GET /generar/ventas, /generar/stock-critico, /generar/pedidos  |
| galeria            | 8088   | GET, GET /{id}, GET /producto/{id}, POST, DELETE                                             |
| gastos             | 8089   | GET, GET /{id}, POST, PUT, DELETE, GET/POST /categorias                                      |
| service_calendario | 8090   | CRUD de producción + PATCH /estado, CRUD de bloqueos                                         |

## 1.5 Manejo de excepciones

Cada microservicio centraliza el manejo de errores mediante una clase anotada con `@RestControllerAdvice` (`GlobalExceptionHandler`), que traduce las excepciones de negocio en respuestas HTTP coherentes:

- `ResourceNotFoundException` → 404 Not Found, cuando se busca un recurso por id que no existe.
- `IllegalArgumentException` → 400 Bad Request, para validaciones de negocio.
- `MethodArgumentNotValidException` → 400 Bad Request, con el detalle de los campos inválidos (validación de `@Valid` sobre los DTOs/entidades).
- `AccessDeniedException` → 403 Forbidden, cuando Spring Security rechaza la petición por falta de privilegios.
- `Exception` (genérica) → 500 Internal Server Error, como respaldo para errores no previstos.

```
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
```

```
ErrorResponse error = new ErrorResponse(  
    HttpStatus.NOT_FOUND.value(), "No encontrado", ex.getMessage());  
return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);  
}
```

Este enfoque evita repetir bloques try/catch en cada Controller y asegura que todos los microservicios respondan errores con un formato JSON consistente (status, error, mensaje).

## 2. Diagrama actualizado

### Diagrama entidad-relación base de datos

El siguiente diagrama representa el modelo de datos completo del sistema, incluyendo las 19 entidades distribuidas entre los distintos microservicios y sus relaciones (claves foráneas lógicas, ya que físicamente cada base de datos es independiente).

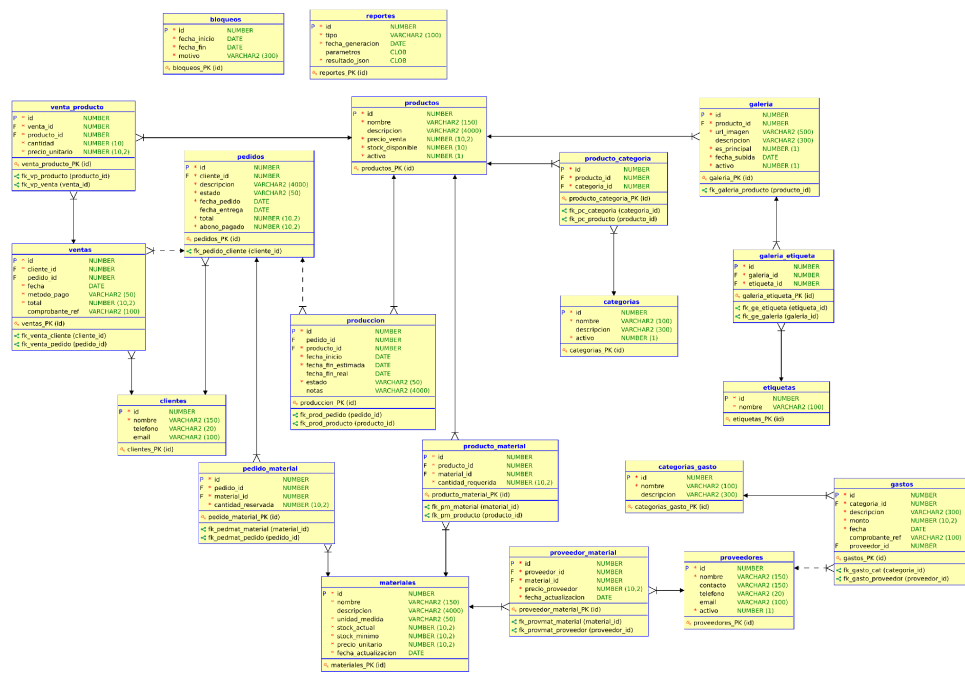
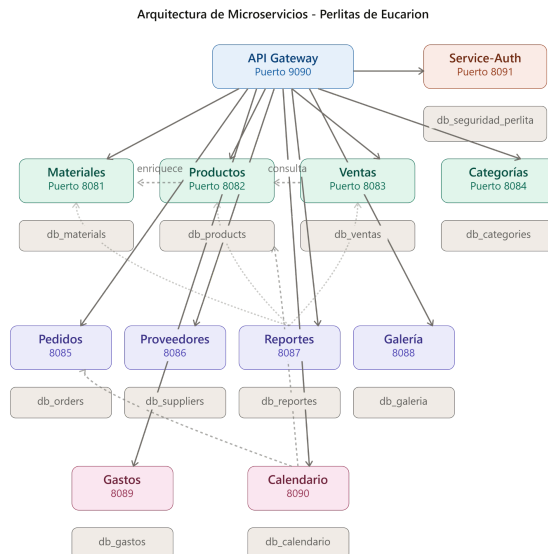


Figura 1. Diagrama entidad-relación del sistema “Perlitas de Eucarion”.

### 2.2 Diagrama de arquitectura de microservicios

El siguiente diagrama debe representar, a nivel de componentes, la arquitectura completa del sistema: el Gateway como punto de entrada único, los 10 microservicios de negocio, el servicio de autenticación, sus bases de datos independientes y las comunicaciones síncronas entre servicios (productos → materiales, ventas → productos, service\_calendario → productos y pedidos, reportes → todos los servicios).



### 3. Servicio de autenticación

La autenticación es el proceso mediante el cual el sistema verifica la identidad de un usuario (a través de su nombre de usuario y contraseña), mientras que la autorización determina qué acciones puede realizar ese usuario una vez identificado. En este proyecto ambos procesos se resuelven mediante un esquema de tokens JWT (JSON Web Token): el usuario se autentica una sola vez contra el microservicio service-auth, y recibe un token firmado que adjunta en cada petición posterior. El sistema define tres roles de negocio:

| Rol      | Permisos                                                                                        |
|----------|-------------------------------------------------------------------------------------------------|
| ADMIN    | Acceso completo a todos los endpoints (lectura, creación, edición y eliminación).               |
| VENDEDOR | Puede consultar, crear y actualizar (GET, POST, PUT), pero no puede eliminar recursos (DELETE). |

#### 3.2 Autenticación por token

El microservicio service-auth (puerto 8091, base de datos db\_seguridad\_perlita) expone dos endpoints públicos: /auth/registrar y /auth/login. La lógica reside en la clase AuthService:

- registrar(AuthRequest request): valida que el nombre de usuario no exista, encripta la contraseña con BCryptPasswordEncoder, asigna el o los roles solicitados (o VENDEDOR por defecto) y persiste el nuevo usuario.
- login(nombreUsuario, contrasena): busca al usuario, valida la contraseña con passwordEncoder.matches(...) y, si es correcta, delega en JwtService la generación del token, incluyendo el nombre de usuario y la lista de roles como claims.

La generación del token se realiza en JwtService, firmando con el algoritmo HS256 y una clave secreta configurada en application.properties (jwt.secret), con una vigencia de 24 horas:

```

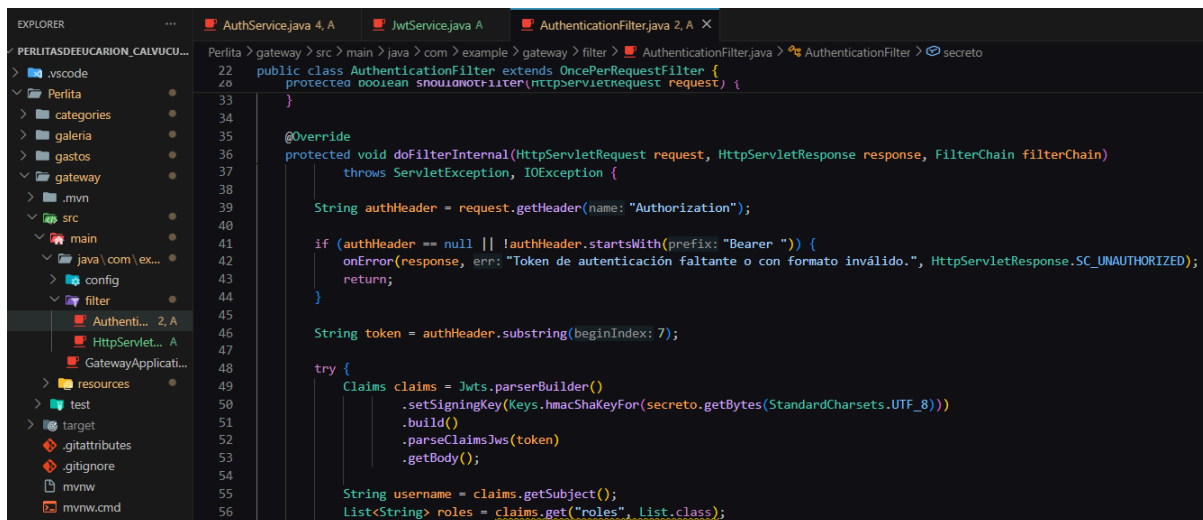
public String generarToken(String username, List<String> roles) {
    long unDiaEnMilisegundos = 1000L * 60 * 60 * 24;
    return Jwts.builder()
        .setSubject(username)
        .claim("roles", roles)
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + unDiaEnMilisegundos))
        .signWith(Keys.hmacShaKeyFor(secreto.getBytes(StandardCharsets.UTF_8)),
SignatureAlgorithm.HS256)
        .compact();
}

```

### 3.3 Validación del token en el Gateway

Todo el tráfico hacia los microservicios pasa primero por el Gateway (puerto 9090), donde un filtro `AuthenticationFilter` (extiende `OncePerRequestFilter`) intercepta cada petición:

- Las rutas `/auth/**`, `/swagger-ui/**` y `/v3/api-docs/**` quedan excluidas del filtro (`shouldNotFilter`), ya que deben ser accesibles sin token.
- Para el resto de las rutas, exige un header `Authorization: Bearer <token>`; si falta o tiene formato inválido, responde 401.
- Si el token está presente, lo valida con la misma clave secreta (`Jwts.parserBuilder()....`); si está expirado o la firma es inválida, responde 401.
- Aplica reglas de autorización adicionales a nivel de Gateway: si el rol es `CLIENTE` y el método no es `GET`, responde 403; si el rol es `VENDEDOR` y el método es `DELETE`, responde 403.
- Si el token es válido, propaga la identidad hacia el microservicio destino agregando los headers `X-User-Username` y `X-User-Roles` a la petición.



```
22 public class AuthenticationFilter extends OncePerRequestFilter {
23     protected boolean shouldNotFilter(HttpServletRequest request) {
24     }
25
26     @Override
27     protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
28         throws ServletException, IOException {
29
30         String authHeader = request.getHeader("Authorization");
31
32         if (authHeader == null || !authHeader.startsWith("Bearer ")) {
33             onError(response, err: "Token de autenticación faltante o con formato inválido.", HttpServletResponse.SC_UNAUTHORIZED);
34             return;
35         }
36
37         String token = authHeader.substring(beginIndex: 7);
38
39         try {
40             Claims claims = Jwts.parserBuilder()
41                 .setSigningKey(Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8)))
42                 .build()
43                 .parseClaimsJws(token)
44                 .getBody();
45
46             String username = claims.getSubject();
47             List<String> roles = claims.get("roles", List.class);
48         }
49     }
50 }
51
52
53
54
55
56
57
```

### 3.4 Spring Security en cada microservicio: protección de rutas

Cada microservicio de negocio no vuelve a validar la firma del JWT (esa responsabilidad es exclusiva del Gateway); en su lugar, confía en los headers `X-User-Username` y `X-User-Roles` reenviados por el Gateway. Un filtro propio, `RequestHeaderAuthenticationFilter`, lee esos headers y construye el contexto de seguridad de Spring:

```
String username = request.getHeader("X-User-Username");
String rolesStr = request.getHeader("X-User-Roles");

List<SimpleGrantedAuthority> authorities = Arrays.stream(rolesStr.split(","))
    .map(rol -> new SimpleGrantedAuthority(rol.trim().toUpperCase()))
    .collect(Collectors.toList());

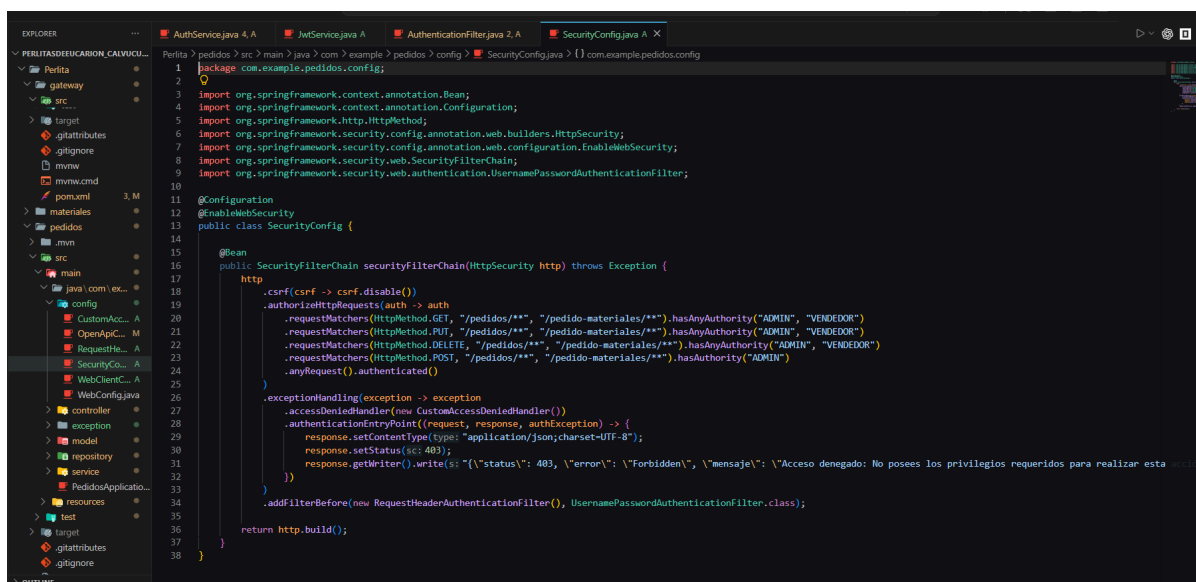
SecurityContextHolder.getContext().setAuthentication(
    new UsernamePasswordAuthenticationToken(username, null, authorities));
```



Sobre ese contexto, cada SecurityConfig define reglas finas por verbo HTTP usando hasAuthority/hasAnyAuthority. Por ejemplo, en el microservicio de materiales:

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.GET, "/materiales/**").hasAnyAuthority("ADMIN", "VENDEDOR")
    .requestMatchers(HttpMethod.PUT, "/materiales/**").hasAnyAuthority("ADMIN", "VENDEDOR")
    .requestMatchers(HttpMethod.POST, "/materiales/**").hasAuthority("ADMIN")
    .requestMatchers(HttpMethod.DELETE, "/materiales/**").hasAuthority("ADMIN")
    .anyRequest().authenticated())
```

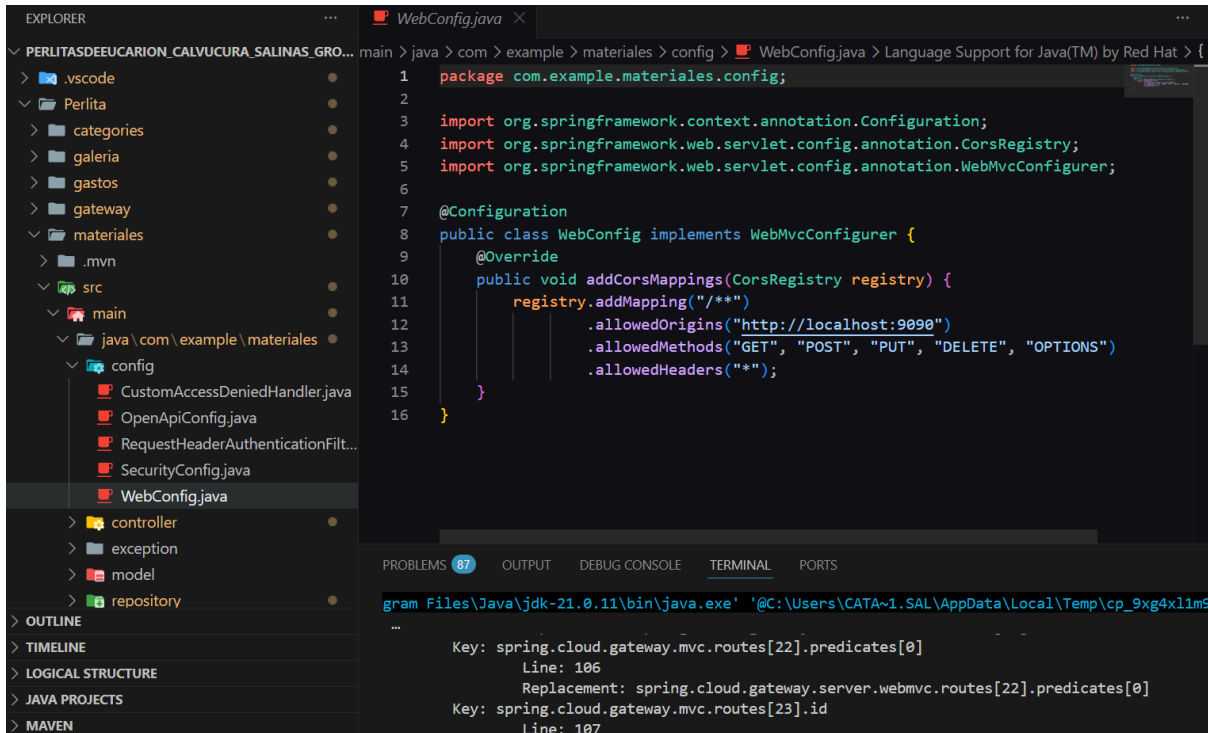
De esta forma la autorización queda implementada en dos capas complementarias: el Gateway aplica reglas generales por rol, y cada microservicio aplica reglas específicas según sus propias necesidades de negocio. Los accesos denegados se devuelven con un cuerpo JSON estandarizado (status, error, mensaje) mediante un accessDeniedHandler y un authenticationEntryPoint personalizados.



### 3.5 CORS

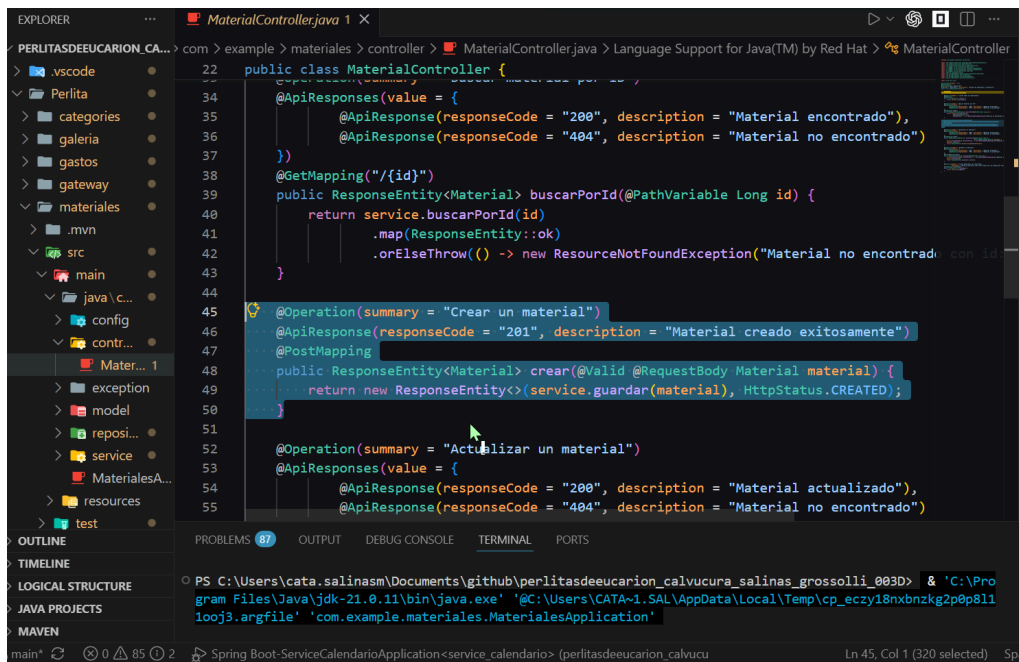
Para permitir que un eventual frontend (servido en otro origen) consuma la API a través del Gateway, cada microservicio define una configuración CORS mediante WebMvcConfigurer, restringiendo el origen permitido al del propio Gateway:

```
@Override
public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/**")
        .allowedOrigins("http://localhost:9090")
        .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
        .allowedHeaders("*");
}
```



### 3.6 Evidencia de funcionamiento (Postman)

A continuación se documentan los cuatro escenarios principales del flujo de autenticación y autorización, verificados con Postman.



HTTP <http://localhost:9090/materiales> Save

POST <http://localhost:9090/materiales> Send

Params Authorization Headers (9) **Body** Pre-request Script Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary JSON Beautify

```
1
2 "nombre": "Hilo de seda",
3 "descripcion": "Hilo fino para pulseras",
4 "unidadMedida": "metros",
5 "stockActual": 100,
6 "stockMinimo": 10,
7 "precioUnitario": 500
```

Body Cookies Headers (14) Test Results 201 Created 4.65 s 620 B Save Response

Pretty Raw Preview Visualize JSON ⋮

```
1
2 {"id": 8,
3  "nombre": "Hilo de seda",
4  "descripcion": "Hilo fino para pulseras",
5  "unidadMedida": "metros",
6  "stockActual": 100.0,
7  "stockMinimo": 10.0,
```

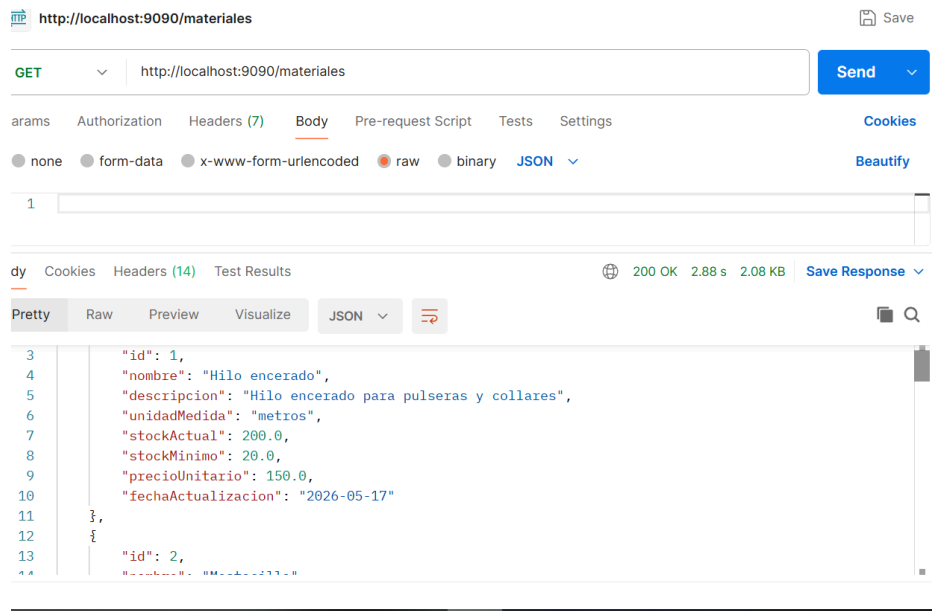
MaterialController.java 1 X

MaterialController.java > Language Support for Java(TM) by Red Hat > MaterialController

```
19 @RestController
20 @RequestMapping("/materiales")
21 @Tag(name = "Materiales", description = "Gestión de materiales e
22 public class MaterialController {
23
24     @Autowired
25     private MaterialService service;
26
27     @Operation(summary = "Listar todos los materiales")
28     @GetMapping
29     public List<Material> listar() {
30         return service.listarTodos();
31     }
32
33     @Operation(summary = "Buscar material por ID")
34     @ApiResponses(value = {
35         @ApiResponse(responseCode = "200", description = "Material
36         @ApiResponse(responseCode = "404", description = "Material
37     })
38     @GetMapping("/{id}")
39     public ResponseEntity<Material> buscarPorId(@PathVariable Long id) {
40         return service.buscarPorId(id)
41             .map(ResponseEntity::ok)
42             .orElseThrow(() -> new ResourceNotFoundException("Mater
```

PROBLEMS (87) OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\cata.salinas\Documents\github\perlitasdeeducarion_calvucura_salinas_g
gram Files\Java\jdk-21.0.11\bin\java.exe' '@C:\Users\CATA~1\SAL\AppData\Local\Ten
100j3.argfile' 'com.example.materiales.MaterialesApplication'
```



## 4. Implementación por capas

### 4.1 El patrón Controller-Service-Repository

Como se introdujo en la sección 1.2, cada microservicio separa sus responsabilidades en tres capas. Esto se ilustra a continuación con el microservicio de Materiales, mostrando el recorrido completo de una petición HTTP desde que llega al Controller hasta que se persiste en la base de datos.

### 4.2 Ejemplo completo: microservicio de Materiales

#### *Capa Repository*

Define el acceso a datos extendiendo JpaRepository, que ya provee las operaciones CRUD básicas (save, findAll, findById, deleteById). Se agrega además una consulta derivada para el caso de uso de stock bajo:

```
@Repository
public interface MaterialRepository extends JpaRepository<Material, Long> {
    List<Material> findByStockActualLessThan(Double stockMinimo);
}
```

#### *Capa Service*

Contiene la lógica de negocio. En este caso es deliberadamente delgada porque las reglas de materiales son simples, pero es aquí donde se actualiza la fecha de modificación antes de guardar, y donde se ubicaría cualquier validación adicional:

```
@Service
public class MaterialService {
    @Autowired private MaterialRepository repository;
```

```

public List<Material> listarTodos() { return repository.findAll(); }
public Optional<Material> buscarPorId(Long id) { return repository.findById(id); }

public Material guardar(Material material) {
    material.setFechaActualizacion(LocalDate.now());
    return repository.save(material);
}

public void eliminar(Long id) { repository.deleteById(id); }
}

```

## Capa Controller

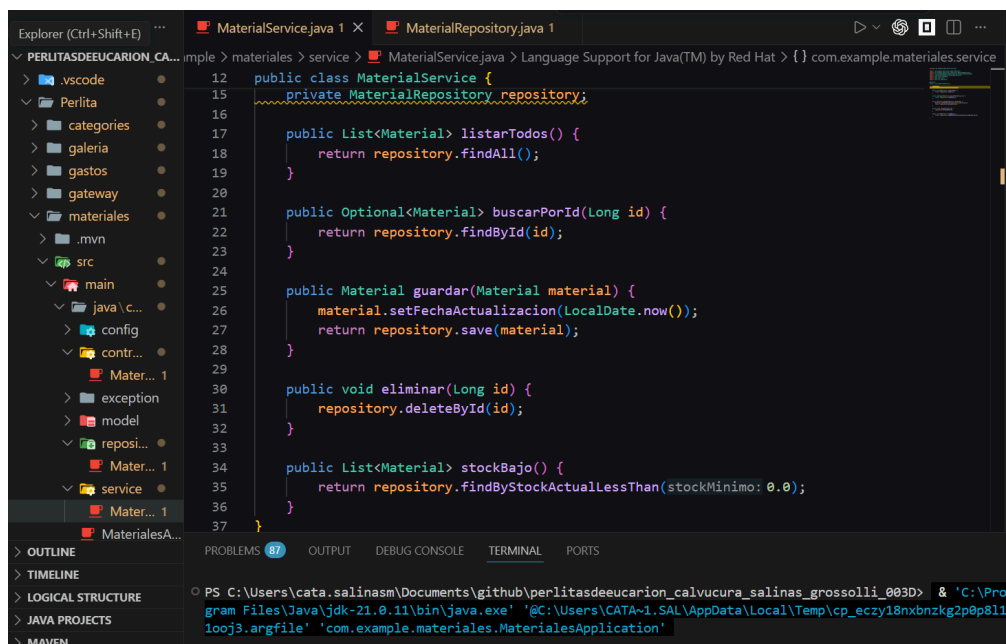
Expone los endpoints REST y traduce las respuestas del Service a códigos HTTP semánticos (200, 201, 204, o lanza `ResourceNotFoundException` para que el `GlobalExceptionHandler` devuelva 404):

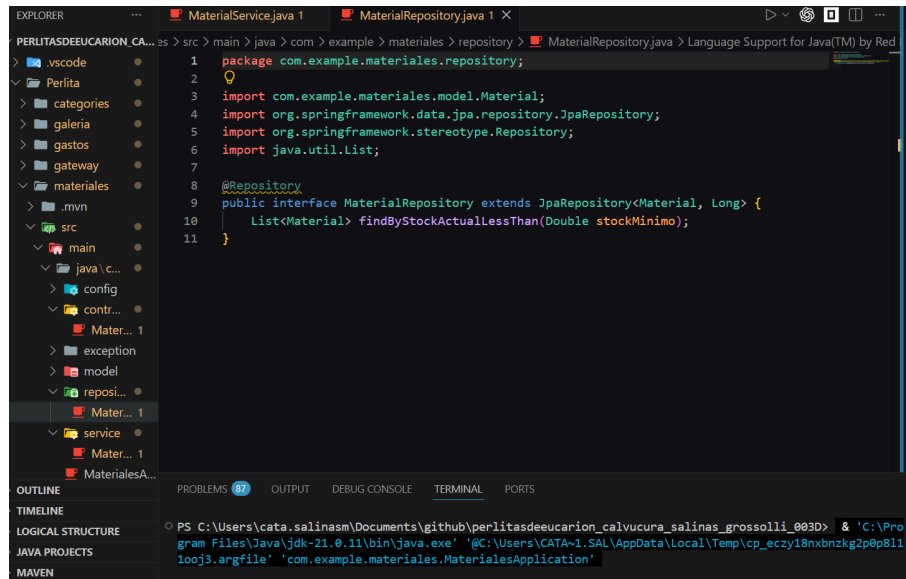
```

@RestController
@RequestMapping("/materiales")
public class MaterialController {
    @Autowired private MaterialService service;

    @PostMapping
    public ResponseEntity<Material> crear(@Valid @RequestBody Material material) {
        return new ResponseEntity<>(service.guardar(material), HttpStatus.CREATED);
    }
    // ... listar(), buscarPorId(), actualizar(), eliminar()
}

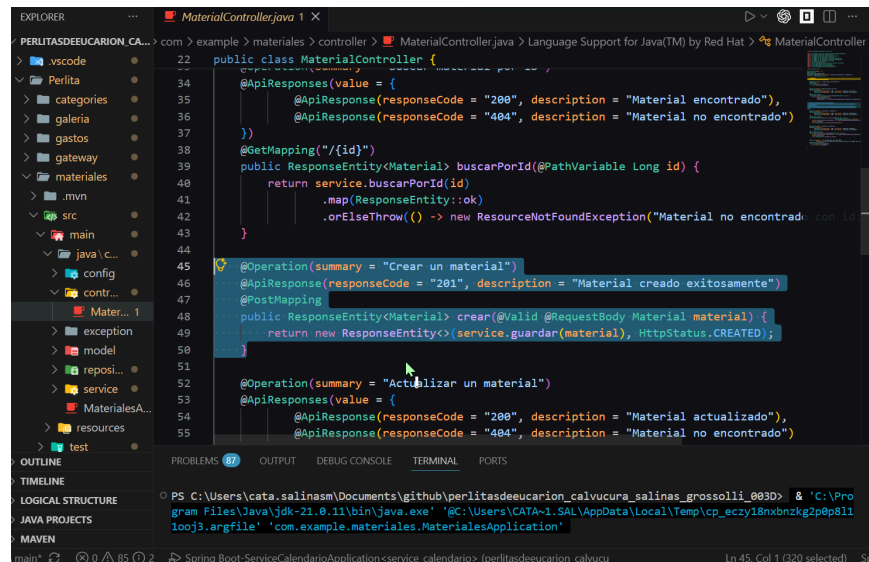
```

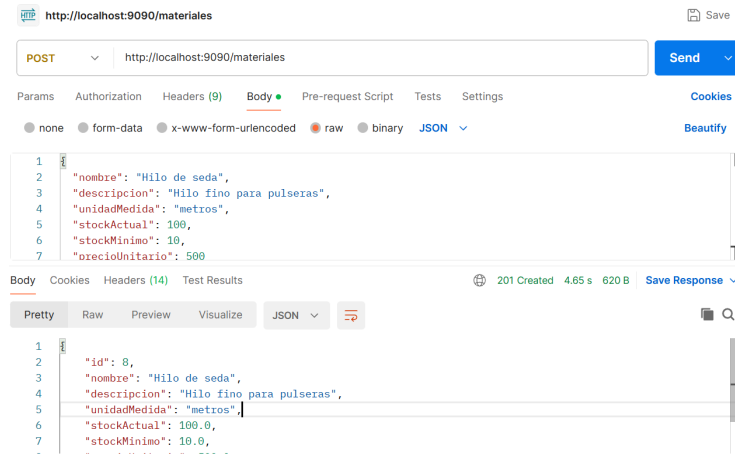


A screenshot of the Visual Studio Code editor. The Explorer panel on the left shows a project structure with folders like 'categories', 'galeria', 'gastos', 'gateway', 'materiales', and 'src'. The 'src' folder is expanded, showing 'main' and 'service' subfolders. The 'main' folder contains 'MaterialRepository.java'. The editor window shows the code for 'MaterialRepository.java'. It starts with a package declaration 'package com.example.materiales.repository;', followed by imports for 'com.example.materiales.model.Material', 'org.springframework.data.jpa.repository.JpaRepository', 'org.springframework.stereotype.Repository', and 'java.util.List'. The class is annotated with '@Repository' and implements 'JpaRepository<Material, Long>'. It has a method 'findByStockActualLessThan(Double stockMinimo)' that returns a 'List<Material>'. The bottom panel shows the terminal with a command to run the application: 'PS C:\Users\cata.salinas\Documents\github\perlitasdeeducacion\_calvucura\_salinas\_grossolli\_003D> & 'C:\Program Files\Java\jdk-21.0.11\bin\java.exe' '@C:\Users\CATA-1.SAL\AppData\Local\Temp\cp\_eczy18nxbnzk2p0p811100j3.argfile' 'com.example.materiales.MaterialesApplication''.

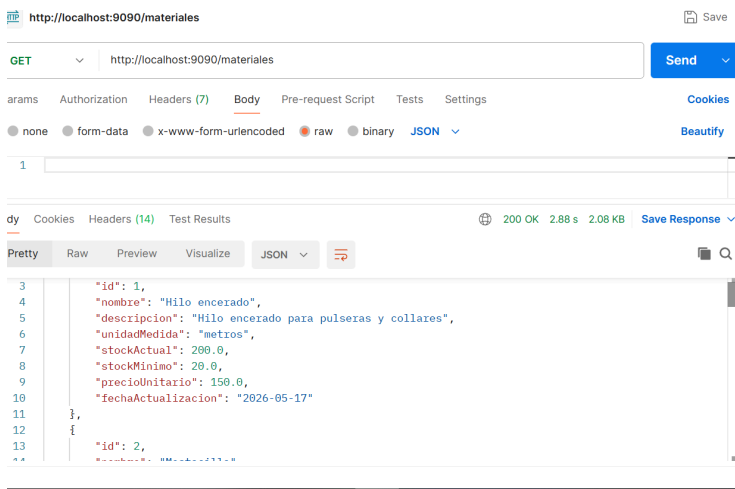
A continuación se documenta, para cada uno de los cinco métodos del CRUD, el código del endpoint junto con su ejecución en Postman.

## POST /materiales — crear

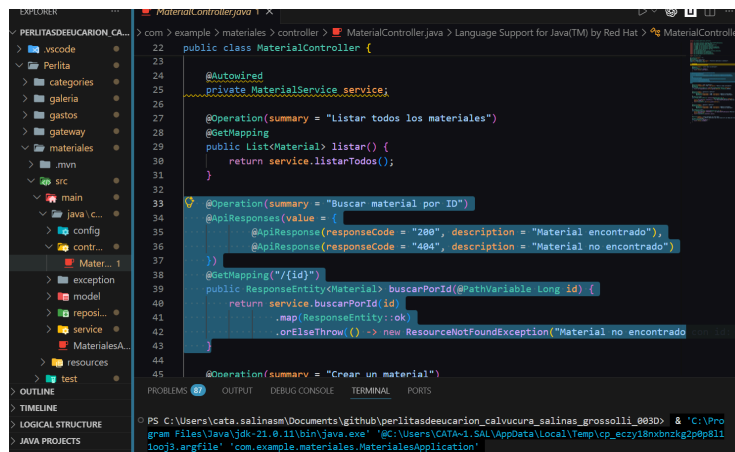
A screenshot of the Visual Studio Code editor showing the 'MaterialController.java' file. The code defines a 'MaterialController' class with two methods. The first method, 'buscarPorId', is annotated with '@GetMapping("/{id}')" and returns a 'ResponseEntity<Material>' by calling 'service.buscarPorId(id)'. The second method, 'crear', is annotated with '@PostMapping' and '@Valid @RequestBody Material material'. It has an '@Operation(summary = "Crear un material")' annotation and returns a 'ResponseEntity<Material>' by calling 'service.guardar(material)' with 'HttpStatus.CREATED'. The bottom panel shows the terminal with the same command as the previous screenshot: 'PS C:\Users\cata.salinas\Documents\github\perlitasdeeducacion\_calvucura\_salinas\_grossolli\_003D> & 'C:\Program Files\Java\jdk-21.0.11\bin\java.exe' '@C:\Users\CATA-1.SAL\AppData\Local\Temp\cp\_eczy18nxbnzk2p0p811100j3.argfile' 'com.example.materiales.MaterialesApplication''.

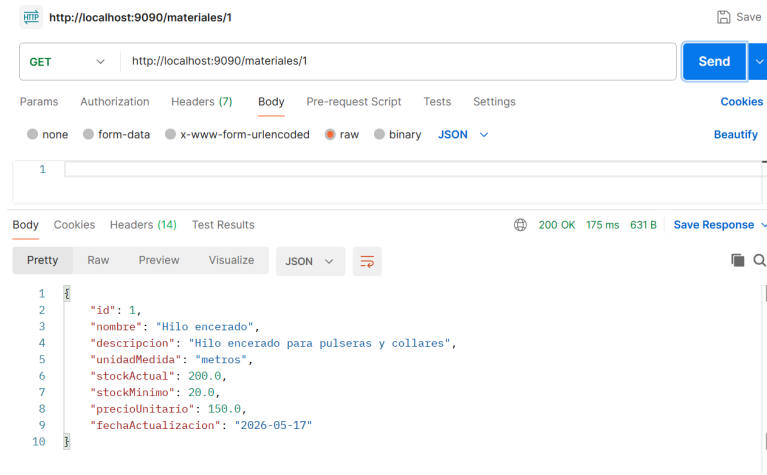


## GET /materiales — listar todos

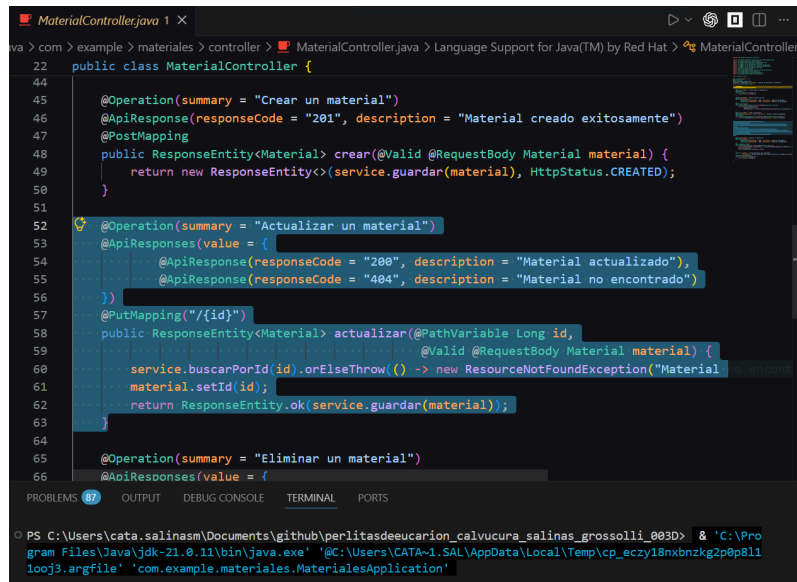


## GET /materiales/{id} — buscar por id

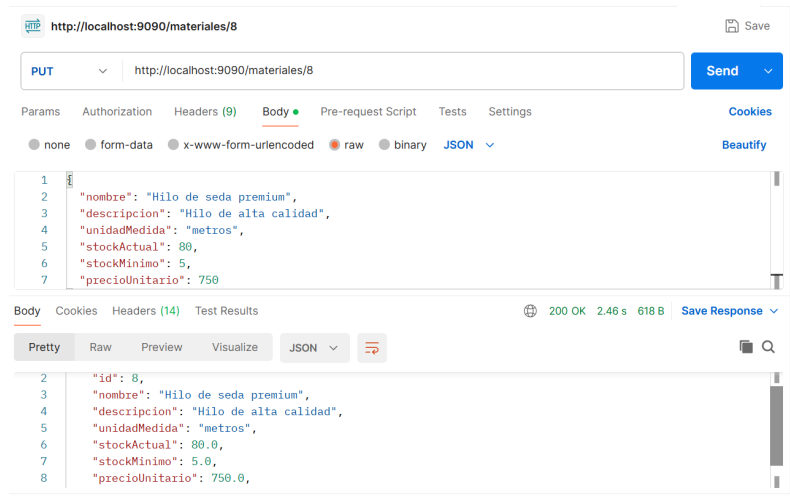




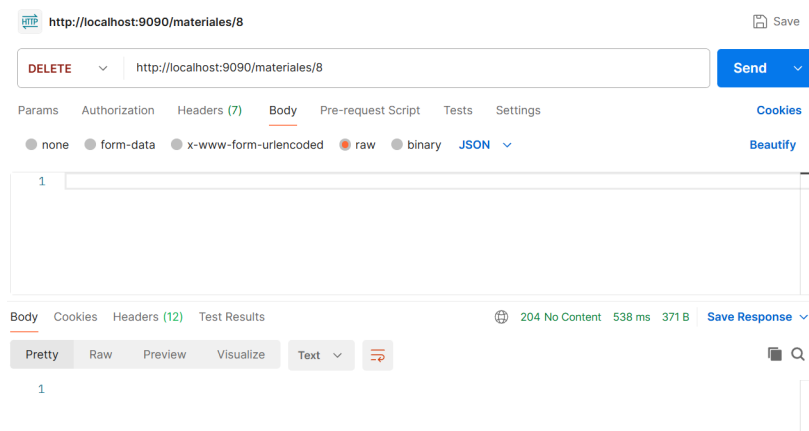
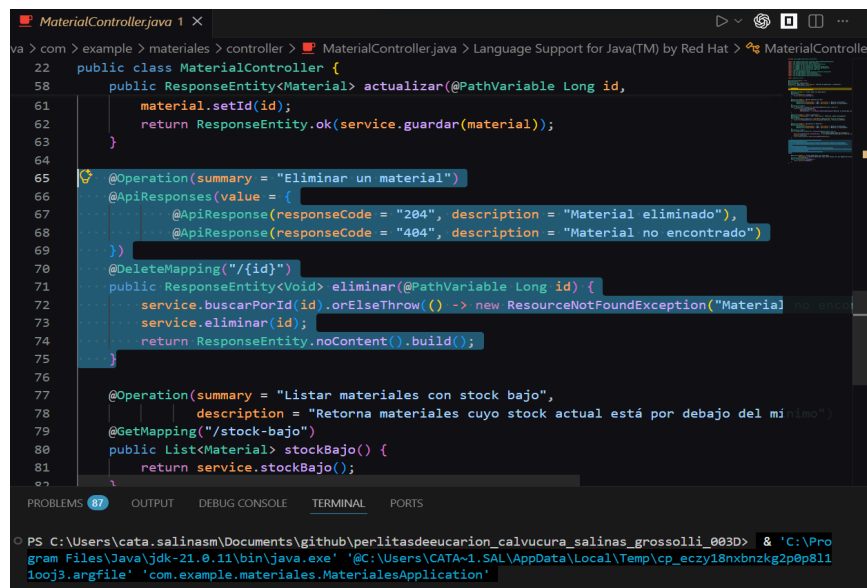
## ***PUT /materiales/{id} — actualizar***







## DELETE /materiales/{id} — eliminar



## 5. Comunicación entre microservicios

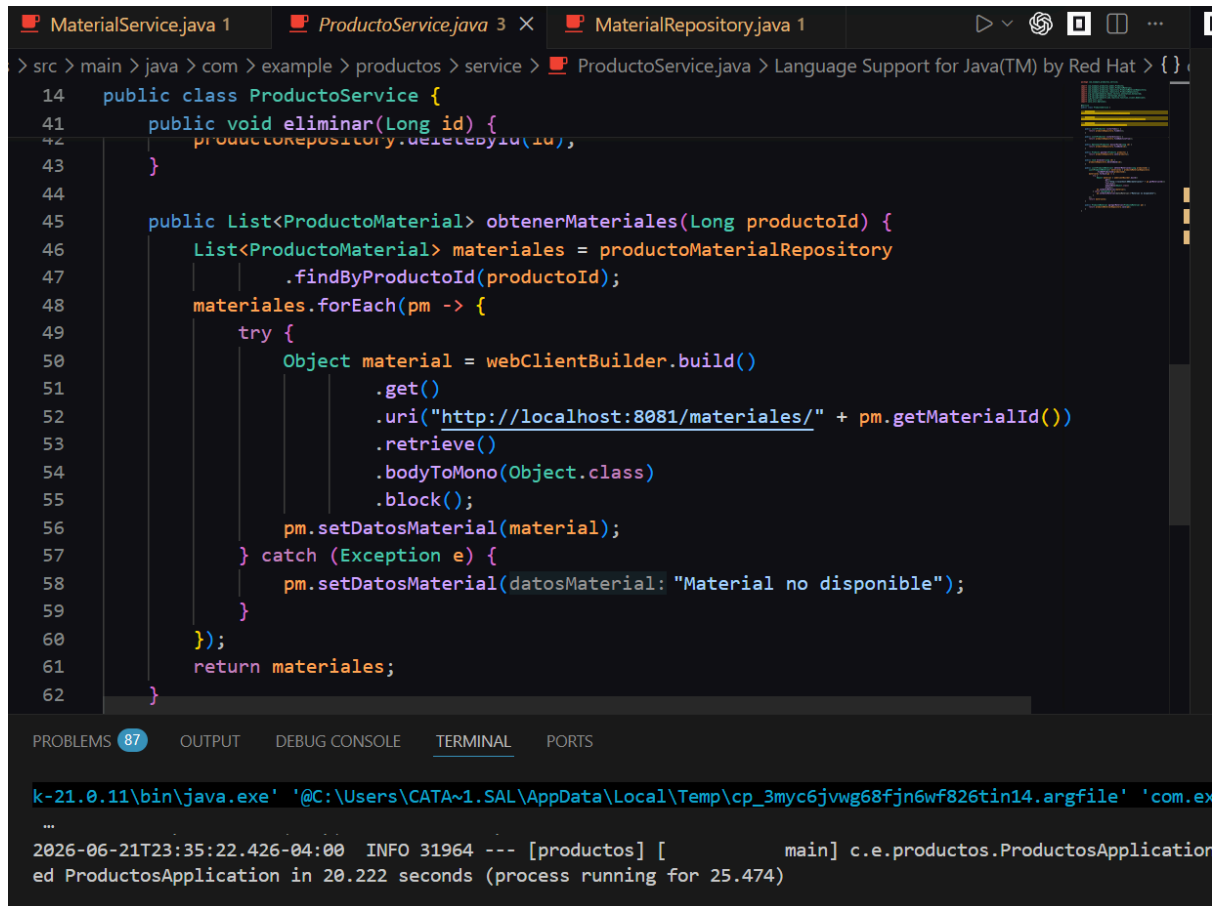
Al tratarse de una arquitectura de microservicios con base de datos independiente por servicio, existen casos donde un microservicio necesita datos que viven en otro. Esto se resuelve mediante comunicación síncrona HTTP usando WebClient (cliente reactivo de Spring WebFlux), en lugar de acceder directamente a una base de datos ajena.

| Servicio origen    | Consulta a                          | Propósito                                                                                 |
|--------------------|-------------------------------------|-------------------------------------------------------------------------------------------|
| productos          | materiales (8081)                   | Enriquecer la información de un producto con los datos de los materiales que lo componen. |
| ventas             | productos (8082)                    | Enriquecer cada línea de venta con los datos del producto vendido.                        |
| service_calendario | productos (8082),<br>pedidos (8085) | Enriquecer la orden de producción con los datos del producto y del pedido asociado.       |
| reportes           | todos los servicios                 | Consolidar información de distintos dominios para generar reportes.                       |

Un ejemplo concreto es el método `obtenerMateriales(productoId)` del microservicio `productos`, que primero obtiene de su propio repositorio la relación producto-material, y luego, por cada material asociado, consulta al microservicio de materiales para traer sus datos actualizados (nombre, stock, precio):

```
public List<ProductoMaterial> obtenerMateriales(Long productoId) {
    List<ProductoMaterial> materiales = productoMaterialRepository
        .findByProductoId(productoId);
    materiales.forEach(pm -> {
        try {
            Object material = webClientBuilder.build()
                .get()
                .uri("http://localhost:8081/materiales/" + pm.getMaterialId())
                .retrieve()
                .bodyToMono(Object.class)
                .block();
            pm.setDatosMaterial(material);
        } catch (Exception e) {
            pm.setDatosMaterial("Material no disponible");
        }
    });
    return materiales;
}
```

La llamada se realiza de forma bloqueante (`.block()`) para simplificar la integración con el flujo síncrono del Controller, y se envuelve en un try/catch para que, si el microservicio de materiales no está disponible, la respuesta de productos no falle por completo sino que indique "Material no disponible". Esto otorga resiliencia básica ante fallos de un servicio dependiente.



```
src > main > java > com > example > productos > service > ProductoService.java > Language Support for Java(TM) by Red Hat > {}

14 public class ProductoService {
41     public void eliminar(Long id) {
42         productoRepository.deleteByid(id);
43     }
44
45     public List<ProductoMaterial> obtenerMateriales(Long productoId) {
46         List<ProductoMaterial> materiales = productoMaterialRepository
47             .findByProductoId(productoId);
48         materiales.forEach(pm -> {
49             try {
50                 Object material = webClientBuilder.build()
51                     .get()
52                     .uri("http://localhost:8081/materiales/" + pm.getMaterialId())
53                     .retrieve()
54                     .bodyToMono(Object.class)
55                     .block();
56                 pm.setDatosMaterial(material);
57             } catch (Exception e) {
58                 pm.setDatosMaterial(datosMaterial: "Material no disponible");
59             }
60         });
61         return materiales;
62     }
}
```

PROBLEMS 87 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
k-21.0.11\bin\java.exe' '@C:\Users\CATA~1\SAL\AppData\Local\Temp\cp_3myc6jvwg68fjn6wf826tin14.argfile' 'com.ex
...
2026-06-21T23:35:22.426-04:00 INFO 31964 --- [productos] [ main] c.e.productos.ProductosApplication
ed ProductosApplication in 20.222 seconds (process running for 25.474)
```

## 6. Swagger

Swagger (OpenAPI) es una especificación y un conjunto de herramientas que permiten documentar de forma automática y estandarizada los endpoints de una API REST: sus rutas, parámetros, cuerpos de petición, posibles respuestas y códigos HTTP. A partir de las anotaciones presentes en el código, genera una interfaz web interactiva (Swagger UI) donde es posible explorar y probar cada endpoint sin salir del navegador, lo que resulta especialmente útil en un proyecto con múltiples microservicios, donde cada uno expone su propio contrato de API.

### 6.2 Implementación de microservicios en Spring Boot

Se incorporó la dependencia `springdoc-openapi-starter-webmvc-ui` en cada microservicio. Esta librería escanea automáticamente los Controllers anotados con `@RestController` y genera la especificación OpenAPI en `/v3/api-docs`, además de exponer la interfaz visual en `/swagger-ui.html`. Dos elementos son necesarios en cada servicio:

- Una clase `OpenApiConfig` que define metadatos generales de la API (título, versión, descripción y el servidor base, en este caso el Gateway).

- Anotaciones `@Tag` a nivel de Controller y `@Operation` / `@ApiResponses` a nivel de cada método, que enriquecen la documentación generada con descripciones legibles para quien consuma la API.

```
@Bean
public OpenAPI customOpenAPI() {
    return new OpenAPI()
        .info(new Info()
            .title("API Perlitas - Servicio de Materiales")
            .version("1.0")
            .description("Gestión de materiales e inventario"))
        .servers(List.of(new
Server().url("http://localhost:9090").description("Gateway")));
}

@RestController
@RequestMapping("/materiales")
@Tag(name = "Materiales", description = "Gestión de materiales e inventario")
public class MaterialController {

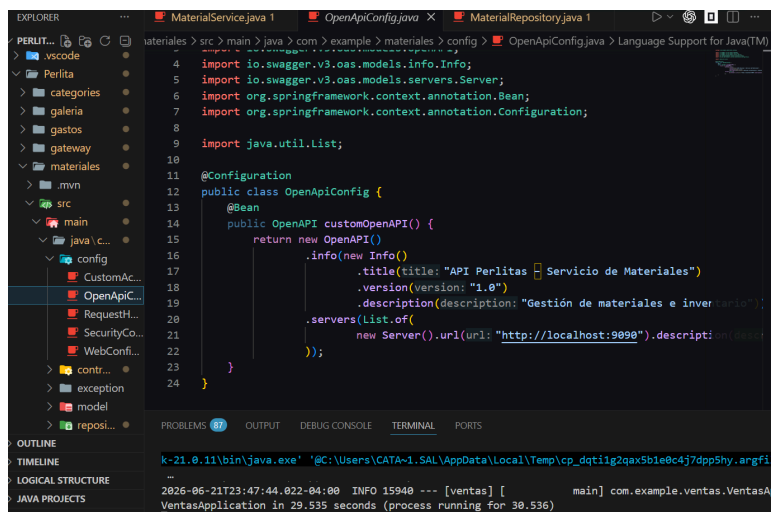
    @Operation(summary = "Buscar material por ID")
    @ApiResponses(value = {
        @ApiResponse(responseCode = "200", description = "Material encontrado"),
        @ApiResponse(responseCode = "404", description = "Material no encontrado")
    })
    @GetMapping("/{id}")
    public ResponseEntity<Material> buscarPorId(@PathVariable Long id) { /* ... */ }
}
```

Adicionalmente, las rutas `/swagger-ui/**`, `/v3/api-docs/**` quedan excluidas tanto del filtro JWT del Gateway como de las reglas de Spring Security de cada microservicio, para que la documentación sea accesible sin necesidad de autenticarse.

## 6.3 Ejemplo en tres microservicios

Por cada uno:

- Captura de código: clase `OpenApiConfig.java`



- Captura de código: un controller con anotaciones `@Tag/@Operation`

```

14 import org.springframework.web.bind.annotation.*;
15
16 import java.util.List;
17
18 @CrossOrigin(origins = "*")
19 @RestController
20 @RequestMapping("/materiales")
21 @Tag(name = "Materiales", description = "Gestión de materiales")
22 public class MaterialController {
23
24     @Autowired
25     private MaterialService service;
26
27     @Operation(summary = "Listar todos los materiales")
28     @GetMapping
29     public List<Material> listar() {
30         return service.listarTodos();
31     }
32
33     @Operation(summary = "Buscar material por ID")
34     @ApiResponse(value = {
35         @ApiResponse(responseCode = "200", description = "Material encontrado"),
36         @ApiResponse(responseCode = "404", description = "Material no encontrado")
37     })
38     public Material buscarMaterialPorID(Long id) {
39         return service.buscarMaterialPorID(id);
40     }
41 }

```

PROBLEMS 87 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

k-21.0.11\bin\java.exe' '@C:\Users\CATA~1\SAL\AppData\Local\Temp\cp_dqti
...
2026-06-21T23:47:44.022-04:00 INFO 15940 --- [ventas] [main]
VentasApplication in 29.535 seconds (process running for 30.536)

```

- Captura de Swagger UI abierto en el navegador mostrando los endpoints de ese microservicio

subir las capturas

## 7. Testing

El testing automatizado consiste en escribir código que ejecuta y verifica el comportamiento de otro código, de forma repetible y sin intervención manual. A diferencia de probar la aplicación “a mano” en Postman, un test queda guardado en el proyecto y puede volver a ejecutarse cada vez que se modifica el código, detectando de inmediato si un cambio rompió una funcionalidad que antes funcionaba (lo que se conoce como regresión).

Testear es especialmente importante en una arquitectura de microservicios como esta, donde múltiples integrantes del equipo modifican distintos servicios de forma simultánea: los tests actúan como una red de seguridad que permite refactorizar o agregar funcionalidad con confianza, documentan el comportamiento esperado de cada método, y reducen la dependencia de pruebas manuales repetitivas en Postman antes de cada entrega.

### 7.2 Pruebas unitarias: el patrón AAA

Las pruebas unitarias implementadas siguen el patrón AAA (Arrange–Act–Assert), que estructura cada test en tres pasos claramente diferenciados:

- Arrange (preparar): se configuran los datos de entrada y el comportamiento simulado de las dependencias (mocks). Por ejemplo, se define qué debe devolver el repositorio cuando se le invoque.
- Act (actuar): se invoca el método que realmente se está probando.
- Assert (verificar): se comprueba que el resultado obtenido sea el esperado, y opcionalmente que las dependencias hayan sido invocadas de la forma correcta (verify()).

Un test unitario de Service aísla la lógica de negocio simulando el Repository con Mockito, de modo que no se necesita una base de datos real para ejecutarlo:

```
@ExtendWith(MockitoExtension.class)
class MaterialServiceTest {
    @Mock private MaterialRepository repository;
    @InjectMocks private MaterialService service;

    @Test
    void buscarPorId_whenExists_shouldReturnMaterial() {
        // Arrange
        Material material = new Material();
        material.setId(1L);
        when(repository.findById(1L)).thenReturn(Optional.of(material));

        // Act
        Optional<Material> actual = service.buscarPorId(1L);

        // Assert
        assertTrue(actual.isPresent());
        assertEquals(1L, actual.get().getId());
        verify(repository).findById(1L);
    }
}
```

## 7.3 Herramientas utilizadas

Se utilizó JUnit 5 como framework de ejecución de pruebas (anotaciones `@Test`, `@ExtendWith`) y Mockito para crear dobles de prueba (mocks) de las dependencias, evitando así levantar el contexto completo de Spring o una base de datos real en cada test de Service. Para los tests de Controller se usó `@WebMvcTest` junto con MockMvc, que permite simular peticiones HTTP contra el Controller de forma aislada, sustituyendo el Service real por un mock con `@MockitoBean`.

```
@WebMvcTest(MaterialController.class)
@AutoConfigureMockMvc(addFilters = false)
class MaterialControllerTest {
    @Autowired private MockMvc mockMvc;
    @MockitoBean private MaterialService service;

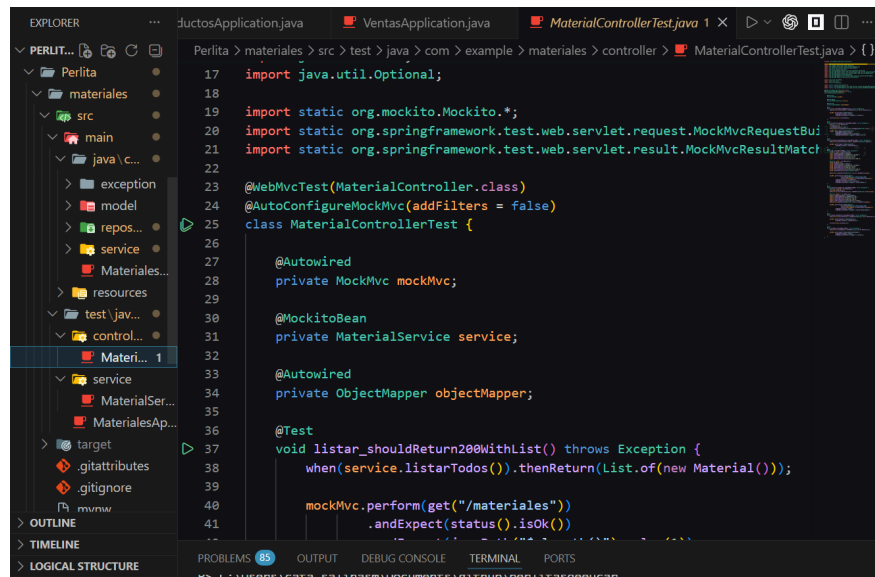
    @Test
    void buscarPorId_whenExists_shouldReturn200() throws Exception {
        // Arrange
        Material m = new Material();
        m.setId(1L); m.setNombre("Test");
        when(service.buscarPorId(1L)).thenReturn(Optional.of(m));

        // Act & Assert
    }
}
```

```

        mockMvc.perform(get("/materiales/1"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.nombre").value("Test"));
    }
}

```



## 7.4 Cobertura de testing del proyecto

La rúbrica exige testear al menos el 60% de los microservicios (6 de 10, sin contar el Gateway). El proyecto cubre con tests de Service y de Controller a 8 de los 10 microservicios de negocio (80%), superando el mínimo exigido. Adicionalmente, el servicio de autenticación cuenta con tests de Service. En total se implementaron 142 métodos de prueba.

| Microservicio            | Test de Service | Test de Controller | Métodos de test |
|--------------------------|-----------------|--------------------|-----------------|
| materiales               | Sí              | Sí                 | 14              |
| productos                | Sí              | Sí                 | 18              |
| pedidos                  | Sí              | Sí                 | 25              |
| categories               | Sí              | Sí                 | 21              |
| proveedores              | Sí              | Sí                 | 16              |
| ventas                   | Sí              | Sí                 | 16              |
| gastos                   | Sí              | Sí                 | 13              |
| galeria                  | Sí              | Sí                 | 12              |
| service-auth (adicional) | Sí              | No                 | 7               |
| reportes                 | No              | No                 | 0               |
| service_calendario       | No              | No                 | 0               |

## Conclusión

La Experiencia 3 consolidó la propuesta de solución para “Perlitas de Eucarion”, completando los 10 microservicios de negocio e incorporando los componentes transversales que una arquitectura de microservicios productiva requiere: un esquema de autenticación y autorización centralizado en el Gateway mediante JWT, con una segunda capa de control de acceso en cada microservicio; documentación de API autogenerada con Swagger/OpenAPI en los servicios principales; y una suite de pruebas unitarias con JUnit 5 y Mockito que cubre el 80% de los microservicios del proyecto, superando el 60% mínimo exigido.

El uso de comunicación síncrona vía WebClient entre servicios permitió mantener la independencia de las bases de datos sin sacrificar la posibilidad de enriquecer la información entre dominios relacionados (por ejemplo, productos con materiales, o ventas con productos). El manejo centralizado de excepciones en cada microservicio, junto con respuestas de error estandarizadas, aporta consistencia a toda la API expuesta a través del Gateway.

Como oportunidades de mejora identificadas durante la elaboración de este informe, se recomienda: incorporar una estructura de branching formal en el repositorio (ramas por funcionalidad o por integrante) para futuras iteraciones; completar el archivo .sql con sentencias de poblamiento (INSERT) además de la definición de esquema; y sumar tests a los microservicios de reportes y service\_calendario para acercar la cobertura al 100%.

## Anexos

---

- Repositorio: [https://github.com/lugrossolli-ux/perlitas\\_de\\_eucarion](https://github.com/lugrossolli-ux/perlitas_de_eucarion)
- README.md del proyecto: instrucciones de instalación, ejecución, endpoints y datos de prueba.
- Diagrama entidad-relación: Relational.png.